SOFRIX INTERVIEW

Full Stack Developer

Muhammad Faizan Yousaf — FAST-NUCES 2026

1 React Native — Complete Guide

Their #1 requirement. Know this cold.

Q: What is React Native? How is it different from React.js?

React.js = builds websites. It renders HTML elements (div, span, button) in the browser.

React Native = builds mobile apps for iOS and Android. It renders native mobile components — not HTML.

Same concept: you write components in JavaScript/JSX. But instead of you use <View>, instead of <p> you use <Text>, instead of <button> you use <TouchableOpacity>.

No CSS files — you use StyleSheet.create({}) to write styles in JavaScript.

React Native acts as a **bridge** between your JS code and the real iOS/Android UI.

Q: JSX vs TSX — what's the difference?

JSX = JavaScript + XML syntax. You write HTML-like tags inside JavaScript. File ends in .jsx or .js

TSX = TypeScript + JSX. Same thing but with TypeScript — meaning you define types for your props and state. File ends in .tsx

Example JSX: `<View style={styles.box}><Text>Hello</Text></View>`

In your HealthMate FYP you used Expo with JSX (JavaScript). TypeScript adds type safety — e.g. you must declare what data a component expects.

If they ask: 'JSX and TSX are the same syntax. TSX just adds TypeScript types on top. I used JSX in HealthMate but I understand TS typing.'

Q: What is Expo? When would you NOT use it?

Expo is a toolchain that wraps React Native. It gives you: easy project setup, pre-built libraries (camera, GPS, notifications), and you don't need Xcode or Android Studio to test.

You used Expo in HealthMate for fast development and device testing.

Don't use Expo when: you need a custom native module Expo doesn't support, you need raw Bluetooth Low Energy access (you worked around this), or you need full control over native iOS/Android code.

In those cases you 'eject' to bare workflow or use plain React Native CLI.

Q: Core React Native components — name them

View — like a div. Container for layout.

Text — the only way to show text. Wraps strings.

Image — displays images from URL or local file.

TextInput — input field for user to type.

TouchableOpacity — button wrapper that fades on press.

ScrollView — scrollable container for content.

FlatList — efficient list for long data (only renders visible items).

StyleSheet — CSS-like object to define styles in JS.

Q: Navigation in React Native — what library, stack vs tab?

Library: **React Navigation** (most common). You install it separately.

Stack Navigator: Think browser back/forward. Login screen → Home screen → Profile screen. Each new screen is pushed on top. Press back = pop it off.

Tab Navigator: Bottom bar tabs — Home, Search, Profile. Switching tabs doesn't destroy the screen, state is preserved.

Drawer Navigator: Side menu that slides in.

In HealthMate: Stack navigator for auth flow (Login → Register → Dashboard), Tab navigator for main app sections (Home, Health Data, Profile).

You can nest them: Tab navigator inside a Stack navigator is very common.

Q: How did you handle wearable integration in HealthMate?

Used **React Native BLE (Bluetooth Low Energy)** library to connect to wearable devices.

Wearable sends data (heart rate, steps) as BLE characteristic values. We read those values, parsed them in JavaScript, then sent them to the Flask backend via REST API.

Data stored in MySQL. App fetched it and displayed real-time health insights.

Authentication: token-based sessions (JWT) so each user only sees their own data.

If pressed for specifics: 'I used react-native-ble-plx library. Scan for devices, connect, discover services, then read/notify characteristics.'

Q: What is useState and useEffect? (Hooks)

useState: Stores data inside a component that can change. When it changes, the component re-renders.

Example: `const [count, setCount] = useState(0)` — count starts at 0. Call setCount(1) to update.

useEffect: Runs code when something happens — component loads, or a value changes.

Example: fetch data from API when component first loads: `useEffect(() => {{ fetchData() }}, [])`

Empty array [] = run once on mount. [userId] = run every time userId changes.

In HealthMate: useEffect to fetch health data on screen load, useState to store and display it.

2 HealthMate FYP — Your Role

They WILL ask: what did YOU specifically build?

Q: Walk me through HealthMate. What was your role?

What it is: A React Native mobile app for health monitoring. Users track lifestyle data, genetic info, and wearable device data.

My role: I built the mobile frontend in React Native with Expo. I designed the screens, navigation flow, and connected the app to the Flask backend.

Specifically I built:

• Authentication screens (login, register) with JWT token handling

• Dashboard screen showing real-time health metrics

• Wearable device pairing flow using BLE

• Data visualization components for health trends

• REST API integration with the Flask + MySQL backend

Stack: React Native + Expo (frontend), Flask + Python (backend), MySQL (database), Docker (containerization).

TIP: Don't say 'we built' for everything. Use 'I built the mobile frontend' — own your part clearly.

Q: What was the hardest technical challenge in HealthMate?

Wearable integration was the hardest part. BLE communication is asynchronous — you scan, connect, then wait for characteristic notifications.

The challenge: managing BLE state in React — connection drops, device not found, data coming in at different rates.

Solution: used useEffect to manage connection lifecycle, useState to store device status, and a retry mechanism on disconnect.

Also had to parse raw BLE byte data into readable health values — heart rate is a specific bit format.

3 MERN Stack

AI Knowledge OS — know schema, JWT, RAG cold

Q: What is MERN? Explain each part.

M — MongoDB. NoSQL database. Stores data as JSON-like documents instead of tables.

E — Express.js. Backend web framework for Node.js. Handles routes, middleware, API logic.

R — React. Frontend UI library. Builds the user interface.

N — Node.js. JavaScript runtime that lets you run JS on the server (not just in browsers).

Together: React talks to Express via REST API. Express connects to MongoDB. All JavaScript — one language across the whole stack.

Q: Explain your MongoDB schema for AI Knowledge OS.

Three main collections:

Users: { _id, email, passwordHash, createdAt }

Projects: { _id, userId, title, description, createdAt } — linked to user via userId

Documents: { _id, projectId, content, embeddingVector[], tags[], createdAt } — linked to project

The embeddingVector field stores the numeric vector (array of floats) from Sentence Transformers so we can do similarity search for RAG.

Used **Mongoose** for schema definition and validation — it enforces data types and required fields.

Q: How does JWT authentication work? Where do you store it?

1. User logs in with email + password.

2. Server checks credentials, then creates a **JWT token** — a signed JSON object containing userId and role.

3. Token is signed with a secret key only the server knows.

4. Client stores the token in **httpOnly cookies**.

5. Every request automatically sends the cookie. Server verifies the signature and reads userId — no database lookup needed.

Why NOT localStorage? localStorage can be read by any JavaScript on the page. If there's an XSS attack (injected script), it steals the token. httpOnly cookies are invisible to JavaScript — only the browser sends them.

Q: What is RAG? How did you implement it?

RAG = Retrieval Augmented Generation.

Problem with plain LLMs: they answer from training data only. They hallucinate. They don't know your specific project docs.

RAG fix: before asking the LLM, retrieve relevant documents from YOUR data, then give those to the LLM as context.

Your implementation in AI Knowledge OS:

1. User types a question

2. Convert question to embedding vector using **Sentence Transformers**

3. Search MongoDB for documents whose embedding vector is most similar (cosine similarity)

4. Take top 3-5 matching documents

5. Send: question + those documents to the LLM

6. LLM answers based on the actual docs — grounded, not hallucinated

Q: What makes an API RESTful?

Stateless: Each request contains everything needed. Server stores no session between requests.

Resource-based URLs: /users/123 not /getUser?id=123

HTTP verbs mean things: GET=read, POST=create, PUT=replace, PATCH=update part, DELETE=remove

Consistent responses: JSON format, proper status codes (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error)

Example: GET /projects — list all. POST /projects — create one. DELETE /projects/123 — delete id 123.

4 Next.js — RentEase

SSR/SSG/CSR, routing, Django integration

Q: SSR vs SSG vs CSR — explain like I'm new to this

CSR (Client-Side Rendering) — old React way:

Browser gets an empty HTML page. JavaScript runs. JavaScript fetches data. Page fills in. Slow first load, not SEO-friendly.

SSR (Server-Side Rendering):

Server builds the full HTML for each request and sends it. Browser gets a ready page instantly. Good for pages with fresh data (user dashboard, property listings). Used in RentEase for tenant and property pages.

SSG (Static Site Generation):

Page is built once at build time. No server work per request. Fastest possible. Used for pages that rarely change (landing page, about page).

Rule of thumb: Login-protected pages = SSR. Marketing pages = SSG. Highly interactive tools = CSR.

Q: How does file-based routing work in Next.js?

No router config needed. The file's location = its URL.

`pages/index.js` → / (homepage)

`pages/about.js` → /about

`pages/properties/index.js` → /properties

`pages/properties/[id].js` → /properties/123 (dynamic — [id] is a variable)

In Next.js 13+ it's the /app folder with the same concept.

In RentEase: /properties/[id].js showed individual property details. The id was read from the URL using useRouter().

Q: How did you connect Django backend with Next.js?

Django REST Framework exposed JSON API endpoints: /api/properties/, /api/agreements/, /api/payments/

Next.js called these using **fetch** or **axios**.

For SSR pages: called the API inside **getServerSideProps** — runs on the server before page loads.

For client interactions: called API directly inside React components on user action.

CORS: Had to configure Django to allow requests from the Next.js domain (django-cors-headers library).

Auth: JWT token sent in the Authorization header: `Authorization: Bearer <token>`

Q: What is the FastAPI microservice in RentEase?

Main Django API handled core business logic (properties, tenants, agreements, payments).

PDF generation (rent agreement documents) was heavy and slow — didn't want it blocking the main API.

Separated it into a **FastAPI microservice**: a separate Python service that only handles PDF generation using **ReportLab**.

When a user requests a PDF agreement, Django calls the FastAPI service, gets the PDF back, returns it to the user.

This is a **microservice pattern** — each service does one thing. Django API stays fast.

5 Databases

PostgreSQL vs MongoDB, indexes, JOINs

Q: PostgreSQL vs MongoDB — when to use which?

PostgreSQL (SQL — relational):

Use when: data has clear relationships (users own properties, properties have agreements, agreements have payments). Need ACID transactions (financial operations — either all happen or none). Schema is fixed and well-defined. Need complex queries with JOINs.

You used it in RentEase — perfect fit for rental finance data.

MongoDB (NoSQL — document):

Use when: data structure varies per document. Storing JSON-like objects. Schema is flexible and evolving. Need to scale horizontally across many servers.

You used it in AI Knowledge OS — documents have varying content, embeddings, tags.

Q: What are indexes? Why do they matter?

Without an index: database reads every single row to find matches. Called a **full table scan**. Slow on large tables.

With an index: database builds a separate sorted structure (B-tree). Like a book's index — jump directly to the right page.

Adding index on a column you search by (e.g. email, tenant_id) makes reads much faster.

Trade-off: Every write (insert/update) must also update the index. So indexes slow down writes slightly.

Rule: Index columns you frequently filter or join on. Don't index every column.

In RentEase: indexed tenant_id and property_id on the agreements table for fast lookups.

Q: Explain a JOIN query you wrote in RentEase.

Needed to show all active agreements with tenant name and property address in one query:

```
SELECT a.id, t.name AS tenant_name, p.address AS property
FROM agreements a
JOIN tenants t ON a.tenant_id = t.id
JOIN properties p ON a.property_id = p.id
WHERE a.status = 'active';
```

INNER JOIN: Only returns rows that have a match in BOTH tables. No match = row excluded.

LEFT JOIN: Returns ALL rows from left table. If no match on right, those columns are NULL.

6 AI Projects

Whisper pipeline, loan prediction, RAG diagnostic

Q: What did you build at Digital Genei internship?

Built an end-to-end **speech-to-text pipeline using OpenAI Whisper**.

Problem: meetings were recorded but key decisions buried in hours of audio.

Solution: Whisper transcribed the audio → NLP summarization extracted key decisions → stored structured summaries.

Reduced manual review time by ~60% across 50+ meeting sessions.

Also implemented **FAISS** (Facebook AI Similarity Search) for semantic search over transcripts — find past meetings by meaning, not just keywords.

Q: What did you build at Skillify Zone internship?

Built a **loan approval prediction system** using Scikit-learn.

Dataset: 10,000+ records of loan applications with features like income, credit score, employment status.

Steps: data cleaning → exploratory analysis → feature engineering (create useful derived features) → train model → tune hyperparameters → evaluate.

Achieved **87%+ accuracy**.

Model: likely Logistic Regression or Random Forest (mention both and say you compared them).

7 Behavioral

Prepare these word-for-word

Q: "You have 2 months internship, we need 2 years."

Say this plainly — no apology:

"I don't have 2 years on a payroll. But I've been building continuously for 2 years — a production mobile app with real device integration as my FYP, two internships where I shipped working systems, and full-stack projects using your exact stack: React Native, Next.js, Node.js, PostgreSQL, MongoDB. I can read your codebase and contribute from week one."

***TIP:** Do NOT start with 'I know I lack experience but...' — that framing loses the room before you finish.*

Q: Walk me through your most complex project.

Use **RentEase** — most layers, most technical depth.

Problem: Rental market in Pakistan is informal — paper agreements, no digital tracking.

What I built: Full-stack SaaS. Next.js frontend, Django REST backend, PostgreSQL database, FastAPI microservice for PDF generation.

Hardest part: Designing the microservice boundary. Kept PDF generation separate so the main API stayed fast. Used ReportLab for templated rent agreement PDFs.

Result: End-to-end rental lifecycle — list property, generate agreement, track payments, send notices.

Q: How do you handle a bug you can't solve?

Step 1: Isolate — reproduce it in the smallest possible case. Cut the code down until only the bug remains.

Step 2: Read error logs carefully. Don't assume — let the error message tell you where to look.

Step 3: Check official documentation and GitHub issues for that library. Most bugs have been seen before.

Step 4: If still stuck after 30-45 minutes, ask a colleague or post a specific question with a minimal reproducible example.

I don't sit on blockers quietly. I flag them early so the team can plan around them.

8 Questions to Ask Them

Ask these — shows you did research on Sofrix

You: "Is this role primarily client project work or an internal product?"

Why: Their site shows client projects (Parrot AI, Per Diem). Know what your day actually looks like.

You: "How does a typical project work here — who handles requirements and design?"

Why: Agencies often dump PM + design + dev all on one person. Know before you join.

You: "Parrot AI and Per Diem are on your portfolio — is that the type of work I'd be doing?"

Why: Shows you researched the company. Tests if the JD matches reality.

You: "What does the first 30 days look like for this role?"

Why: Shows you're thinking about delivery. Also reveals if they have onboarding or just throw you in.

You: "Is the team here in Faisalabad building for US clients?"

Why: Their website says Santa Monica CA. You're likely the offshore dev team. Know the dynamic.

One rule: Own your FYP work. Say 'I built' not 'we built' for the parts you did. You know this stack — speak with confidence.